

Vasic-Digital-Reusable-Module-Suite

و Go یک بار بسازید، همه جا استفاده کنید — ناوگانی از ماژول‌های کوچک، مستقل و تست‌شده KMP

خلاصه

خانواده‌ای بزرگ از ماژول‌های عمومی و قابل استفاده مجدد که تحت فضاهای نامی منتشر شده‌اند. هر ماژول به صورت Kotlin Multiplatform و `digital.vasic.* (Go)` مستقل، تست‌شده و نسخه‌بندی‌شده است و به عنوان زیرماژول هم‌کد در محصولات بزرگ‌تر مورد استفاده قرار می‌گیرد. این صفحه (و ناوگان گسترده‌تر Catalogizer، HelixAgent) مجموعه‌ای از ابزارهای کوچک متعدد را که به صورت جداگانه تنها به عنوان نویز ظاهر می‌شدند، یکجا گردآوری کرده است.

توضیح کوتاه

اجزای زیرساختی پایه (احراز — `digital.vasic.*` مجموعه‌ای منتخب از ماژول‌های مستقل RAG، عامل/AI هویت، حافظه موقت، پایگاه داده، پیکربندی، قابلیت مشاهده)، بلوک‌های ساخت تیم قرمز) LLM-عامل‌گرا، برنامه‌ریزی و محافظ‌های تدافعی، MCP، پایگاه داده برداری، تعبیه‌ها هر یک از این ماژول‌ها Kotlin Multiplatform. (نرمال‌سازی) — به علاوه مجموعه‌ای آینه‌ای از عمومی، تست‌شده و قابل استفاده مجدد هستند.

توضیح مفصل

سازمان واسیک-دیجیتال بر یک شرط ساختاری استوار است: فلسفه‌ای مبتنی بر «قانون اساسی به علاوه ماژول‌های مستقل و قابل استفاده مجدد متعدد» که در آن هر قابلیت عمومی هرگز دو بار نوشته نمی‌شود. به جای مونولیت‌ها، هر نگرانی قابل استفاده مجدد به ماژول کوچک خود — مخزن، تست‌ها و مستندات مستقل — تبدیل می‌شود و به شدت مستقل نگه داشته می‌شود تا هیچ جزئیاتی از مصرف‌کننده به آن نفوذ نکند. این صفحه این ماژول‌ها را گردآوری کرده است زیرا هر یک به تنهایی در مقیاس یک کتابخانه است و به صورت جداگانه تنها به عنوان نویز در صفحه محصول ظاهر می‌شد. اما در مجموع، نیروی واقعی ضرب‌کننده سازمان هستند: درایی مهندسی خصوصی که «ساخت یک محصول جدید» را به «سرمبندی قطعات اثبات‌شده» تبدیل می‌کند و پشتوانه ملموس ادعای این ناوگان است که چرخ را دوباره اختراع نمی‌کند — بلکه یک چرخ بسیار خوب را نگه می‌دارد و همه جا به کار می‌برد.

لوله‌کشی مورد نیاز هر سرویس (Go) اجزای زیرساختی پایه. این مجموعه سه خوشه را در بر می‌گیرد، مهاجرت‌ها (database)، cache (Redis/TTL)، auth (JWT/bcrypt)، middleware، observability (Prometheus/OpenTelemetry)، ratelimiter، security، storage (S3/MinIO)، streaming (هاب WebSocket)، eventbus، filesystem (چندپروتکله)، discovery/mdns، http3، recovery، concurrency، lazy و موارد دیگر بلوک‌های ساخت. rag، vectordb، embeddings، memory، conversation (فشرده‌سازی زمینه نامحدود، رویدادمحوری) (Go) عامل AI/ (مهاجرتی)، mcp (Model Context Protocol)، toolschema، skillregistry، agentic (درخت فکر/HiPlan/MCTS)، benchmark (SWE-bench/HumanEval/MMLU)، llmops، selfimprove (مدل‌سازی پاداش) و ابزارهای مقاوم‌سازی در برابر LLM-محافظه‌های تدافعی. (نشان‌گذاری شیء‌محور توکن) toon، Normalize (سناریوهای تهاجمی هدایت‌شده توسط) RedTeam: حملات را ارائه می‌دهند. نیز Kotlin Multiplatform مجموعه‌ای موازی از (یکنواخت‌سازی ورودی‌های تهاجمی) (Auth-KMP، Database-KMP، Security-KMP، UI-KMP اجزای و غیره) را برای برنامه‌های چندسکویی فراهم می‌کند.

محتوا

چرا آن را ساختیم

از صفر (و غیره Catalogizer، HelixAgent، Herald) هر بار ساخت تعداد زیادی محصول، اتلاف منابع و ناهماهنگی به همراه دارد. استخراج هر نگرانی عمومی به یک ماژول مستقل و تست‌شده به این معنی است که اصلاحات و بهبودها در کل مجموعه منتشر می‌شوند و هر محصول جدید از قطعات اثبات‌شده سرهم می‌شود.

چرا انقلابی است

— است AI در واقع، این یک «کتابخانه استاندارد» خصوصی برای ساخت بک‌اندهای متمرکز بر لایه‌ای که بیشتر تیم‌ها هرگز فرصت ساخت آن را پیدا نمی‌کنند، زیرا بیش از حد مشغول حل مجدد، برای پنجمین بار هستند. در اینجا، زیرساخت‌های پایه RAG مسائل احراز هویت، کش، و لوله‌کشی همگی به صورت ماژول‌های آماده‌به‌کار و مستقلاً LLM و ریل‌های محافظتی، AI، بلوک‌های سازنده تست‌شده وجود دارند. همین ویژگی است که به یک تیم کوچک امکان می‌دهد سیستم‌های درجه تولید را با سرعتی عرضه کند که معمولاً به تیمی بسیار بزرگ‌تر نیاز دارد، و این کار را بدون انباشت بدهی، تکراری که معمولاً در پی آن می‌آید، انجام دهد.

نوآوری‌ها

- زیرماژول‌ها به عنوان پایگاه‌های کد برابر (CONST-051) نظم جداسازی در سطح مجموعه تلقی می‌شوند و هرگز جزئیات مصرف‌کننده را حمل نمی‌کنند.
- AI (RAG، VectorDB، Embeddings، MCP، ToolSchema، Agentic، Planning، LLMops) لایه اختصاصی پرایمیتیوهای به صورت ماژول‌های قابل استفاده مجدد.
- برای استحکام در برابر LLM (RedTeam، Normalize) خوشه ریل‌های محافظتی حملات خصمانه.
- با قراردادهای مشترک Kotlin Multiplatform و Go مجموعه ماژول‌های موازی.

چالش‌ها و راه‌حل‌ها

- با قرارداد جداسازی قانون اساسی و تزریق: **جلوگیری از پوسیدگی وابستگی** در میان ده‌ها ماژول زمان اجرا برای جزئیات مصرف‌کننده حل شد
- با یک قرارداد مشترک (تست‌ها/مستندات/چالش‌ها: **حفظ ثبات و تست‌پذیری** بسیاری از ماژول‌ها حل شد **HelixConstitution** برای هر ماژول) و ستون فقرات حکمرانی
- حل شد **Kotlin Multiplatform** با آینه‌ای از ماژول‌های اصلی در: **دسترسی چندپلتفرمی**

پشته فناوری (چرایی و چگونگی)

- **Go** — اکثر ماژول‌ها (`digital.vasic.*`).
- **Kotlin Multiplatform** — (احراز هویت/پایگاه داده (KMP)-امنیت/ابط کاربری/همزمانی/محدودکننده نرخ
- **Redis / PostgreSQL / SQLite** — پایگاه داده و ذخیره‌سازی
- **Prometheus / OpenTelemetry** — ماژول مشاهده‌پذیری
- **WebSocket / HTTP/3 (quic-go) / mDNS** — ماژول‌های شبکه
- **Vector DB / embeddings / RAG / MCP** — AI. ماژول‌های پرایمیتیو
- **YAML** — **RedTeam** تنظیمات و فیکسچرهای خصمانه

تأیید نشده / در حال توسعه: چندین مخزن سازمانی با برچسب «چارچوب اولیه / در حال توسعه» مشخص شده‌اند (مانند `PliniusCommon`, `I-LLM`, `HyperTune`, `AutoTemp`, `Veritas`, `Ouroborous`, `Claritas`, `LeakHub`, `GandalfSolutions`). این موارد را به صورت مرحله اولیه/چارچوب اولیه معرفی کنید، نه به عنوان محصول نهایی عرضه شده